PROCESS MANAGEMENT

Unit 2

Topics to be covered

2.1 Introduction: Process vs Program, Multiprogramming, Process Model, Process States, Process Control Block/*Process Table*.

2.2 Threads: Definition, Thread vs Process, *Thread Usage*, User and Kernel Space Threads.

2.3 Inter Process Communication: Definition Race Condition, Critical Section

2.4 Implementing Mutual Exclusion: Mutual Exclusion with Busy Waiting (Disabling Interrupts, Lock Variables, Strict Alteration, Peterson's Solution, Test and Set Lock), Sleep and Wakeup, Semaphore, Monitors, Message Passing

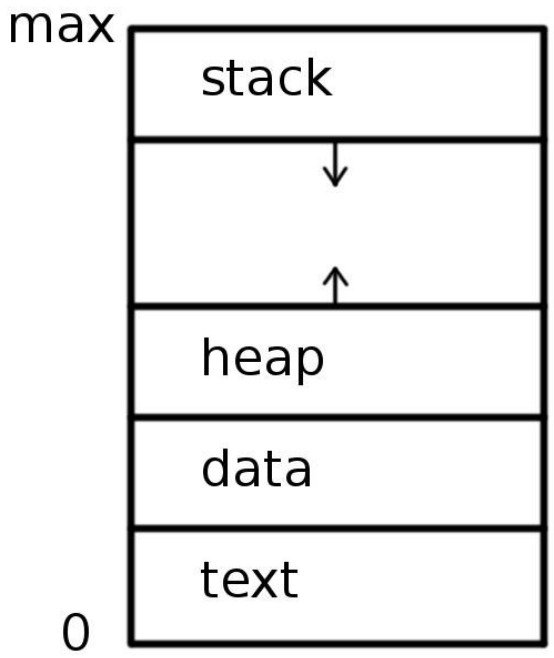
2.5 Classical IPC problems: Producer Consumer, Sleeping Barber, and Dining Philosopher Problem

2.6 Process Scheduling: Goals, Batch System Scheduling (First-Come First-Served, Shortest Job First, Shortest Remaining Time Next), Interactive System Scheduling (Round-Robin Scheduling, Priority Scheduling, Multiple Queues), Overview of Real Time System Scheduling (*No need to discuss any real time system scheduling algorithm*)

Unit 2.1 Introduction

Process

- A "process" refers to anything that is currently running in your system.
- For example, if you click on the browser, say "Google Chrome," the operating system processes this application to open it and keep it open to allow the user to do their task, such as using a search engine to search for something specific or allowing them to do other things in the browser. A "process" is essentially an instance of a software, application, or program that is currently running on your computer system.



Process Vs Program

- Do it yourself.

What is Process Management in OS?

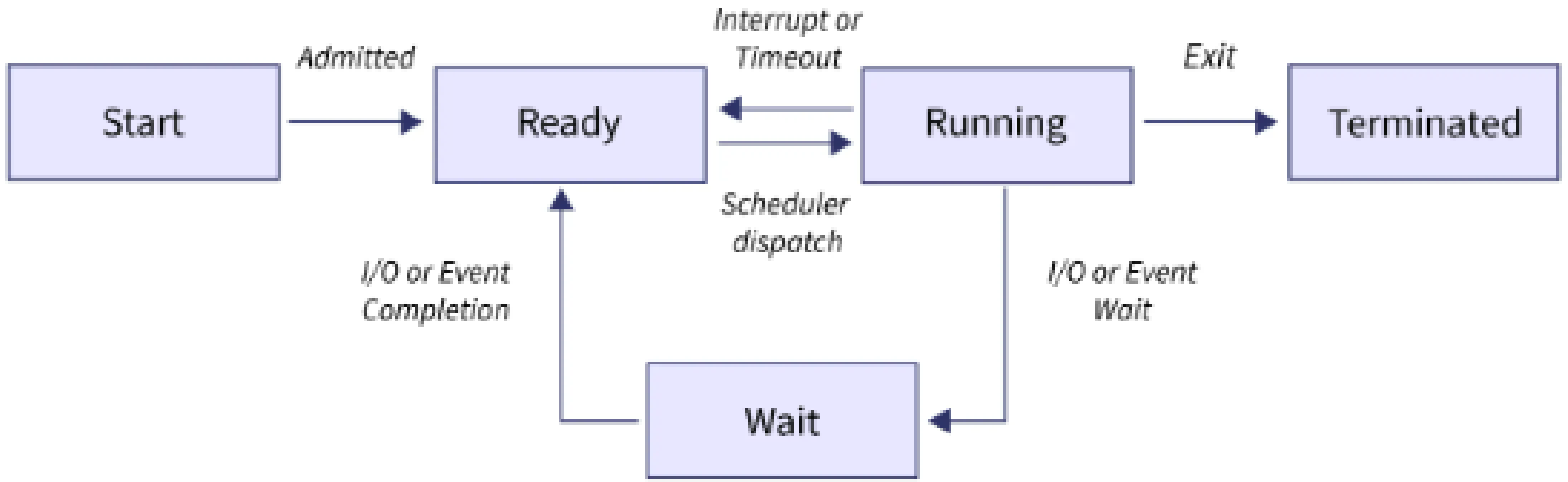
- There are several processes in the process management system that use the same shared resource. As a result, the operating system must efficiently and effectively manage all activities and structures.
- To preserve consistency, some elements may need to be executed by one operation at a time. Otherwise, the system might be inconsistent and a deadlock may develop.
- In terms of Process Management, the operating system is in charge of the following tasks.
  - Process and thread scheduling on the CPU.
  - Both user and system processes can be created and deleted.
  - Processes are suspended and resumed.
  - Providing synchronization methods for processes.
  - Providing communication mechanisms for processes.

Characteristics of a Process

- Process Id: A unique identifier assigned by the operating system
- Process State: Can be ready, running, new , terminate, waiting block.
- CPU registers: Like the Program Counter (CPU registers must be saved and restored when a process is swapped in and out of the CPU)
- Accounts information: Amount of CPU used for process execution, time limits, execution ID, etc
- I/O status information: For example, devices allocated to the process, open files, etc
- CPU scheduling information: For example, Priority (Different processes may have different priorities, for example, a shorter process assigned high priority in the shortest job first scheduling)

Process Life Cycle

- A process state is the status of the process at a given point in time. It also specifies the process current state.



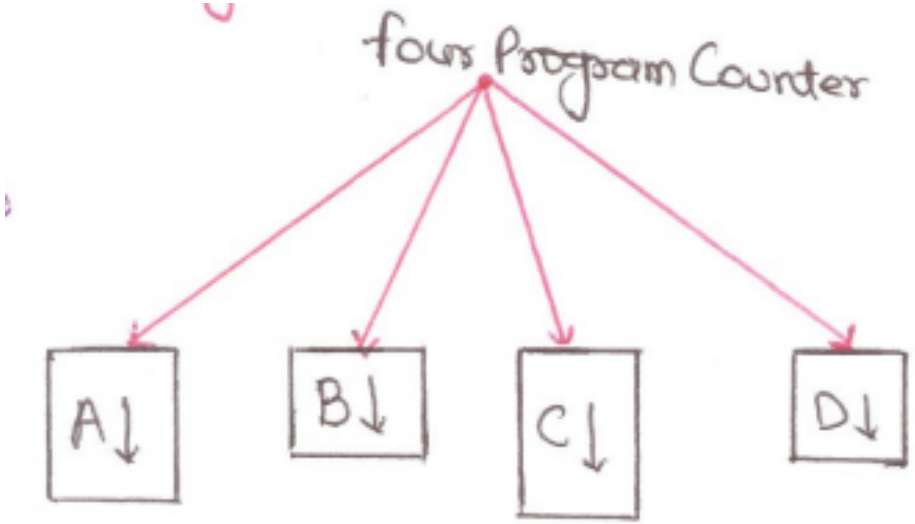
Process Model

- In the process model, all the runnable software on the computer is organized into a number of sequential processes. Each process has its own virtual Central Processing Unit (CPU).
- it is critical that process modeling be implemented in your operating system so that all processes in your system can run smoothly.
- Mainly three Types of Process Models:
  1. Multiprogramming: The real Central Processing Unit (CPU) switches back and forth from process to process. This work of switching back and forth is called multiprogramming.
  2. Multiprocessing
  3. One Program at one time

Multiprogramming

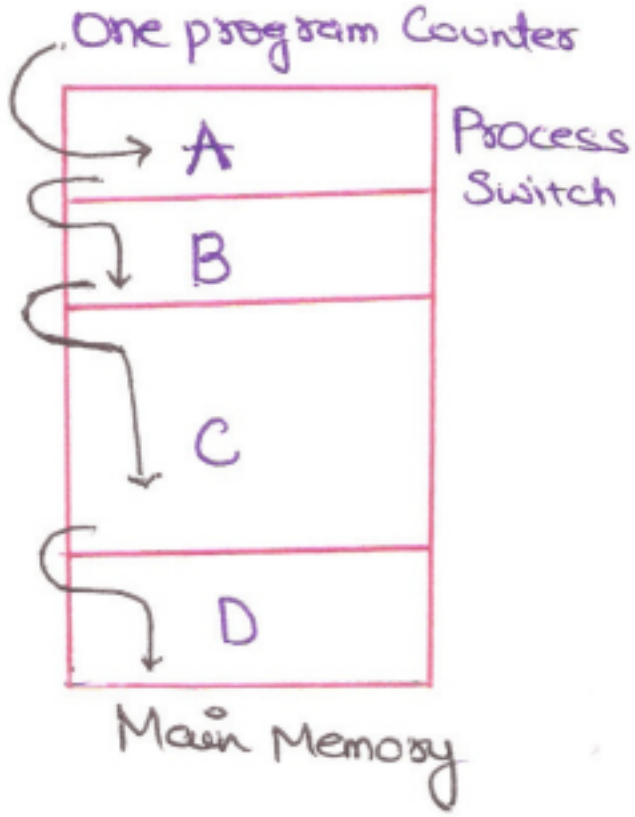
- In any multiprogramming system, the CPU switches from process to process quickly, running each for tens or hundreds of milliseconds. This rapid switching back and forth of the CPU is called multiprogramming.
- In multiprogramming systems, process are performed in a pseudo-parallelism as if each process are performs in a processor.
- In fact there is only one processor but it switches back and forth from process to process.
- Keeping track of multiple parallel activities is hard for people to do. Therefore, OS designers over the years have evolved a sequential process that makes parallelism easier to deal with.

- There is only one program counter all programs in memory.
- In this program counter is initialized to execute the part of program A then with the help of process switch it transfer control to program B and execute some part of this.
- After that program counter for program C is active and execute some part of it and so onto program D.
- Then all the steps continuous may be randomly with the help of process switches until all program is not completed.



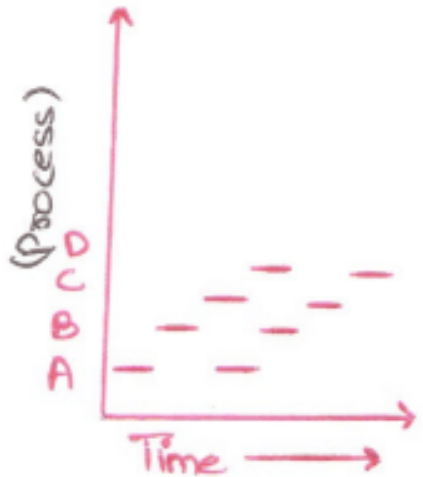
Multiprocessing

- There is 4 processes, each with it own flow of control(that is it on logical program counter), and each when running independently of the other ones.
- There is only one physical program counter. So when each process runs, its logical program counter is loaded into the real program counter.
- When it is finished(for the time being), the physical program counter is saved in the process stored logical program counter in memory.



One Process at One Time

- At any given instant only one process run and other process after some interval of time.
- In other point of view all processes progress but only process actually runs at given instant of time.



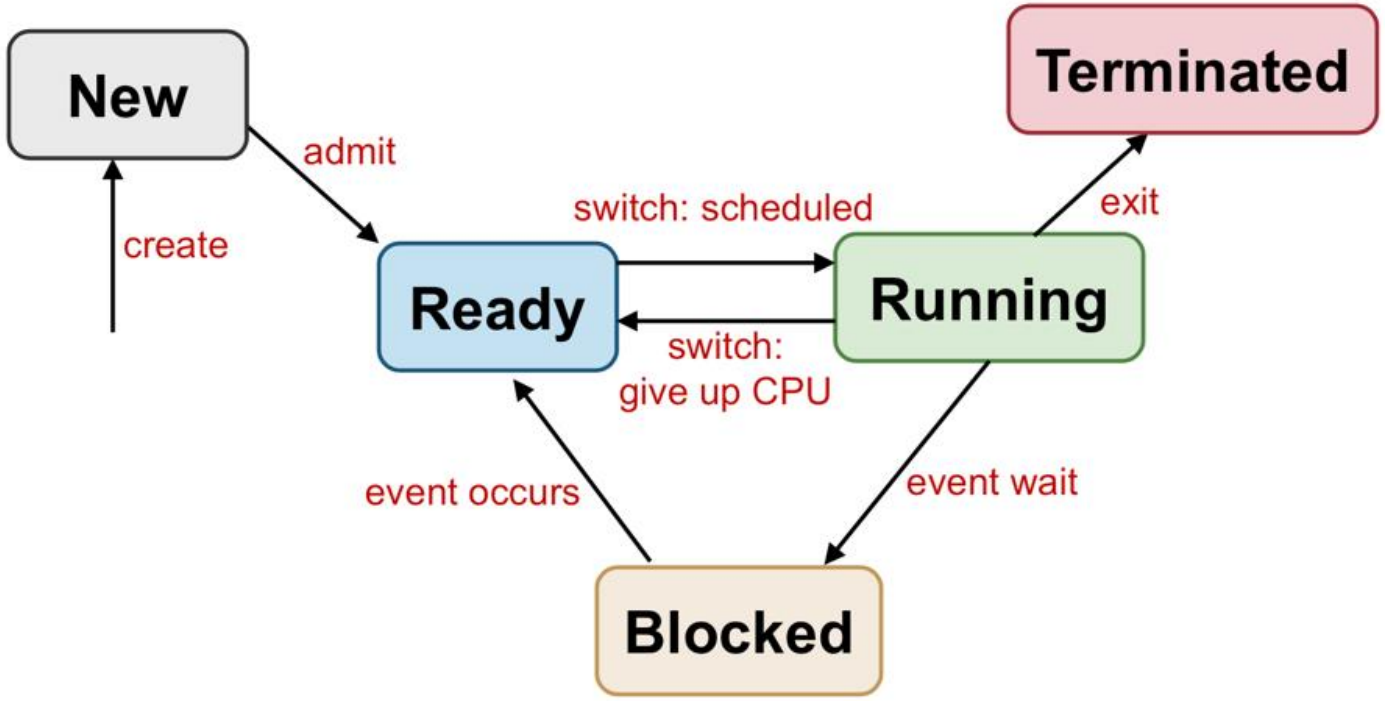
The five process states:

1. **Start-** This is the initial state when a process is first started / created.
2. **Ready -** The process is Waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after start state or while running it by but interrupted by the scheduler to assign CPU to some other process.
3. **Running -** After Ready state, the process state is set to running and the processor executes its instruction.
4. **Waiting-** Process moves into the waiting state if it needs to wait for a resources, such as waiting for user input, or waiting for a file to become available.
5. **Terminated or Exit-** Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it is waits to be removed from main memory.

Execution of Process in Five-state Model

- This model consists of five states i.e, running, ready, blocked, new, and exit.
- The model works when any new job/process occurs in the queue, it is first admitted in the queue after that it goes in the ready state.
- Now in the Ready state, the process goes in the running state.
- In the running state, a process has two conditions i.e., either the process goes to the event wait or the process gets a time-out.
- If the process has timed out, then the process again goes to the ready state as the process has not completed its execution.

- If a process has an event wait condition then the process goes to the blocked state and after that to the ready state.
- If both conditions are true, then the process goes to running state after dispatching after which the process gets released and at last it is terminated.

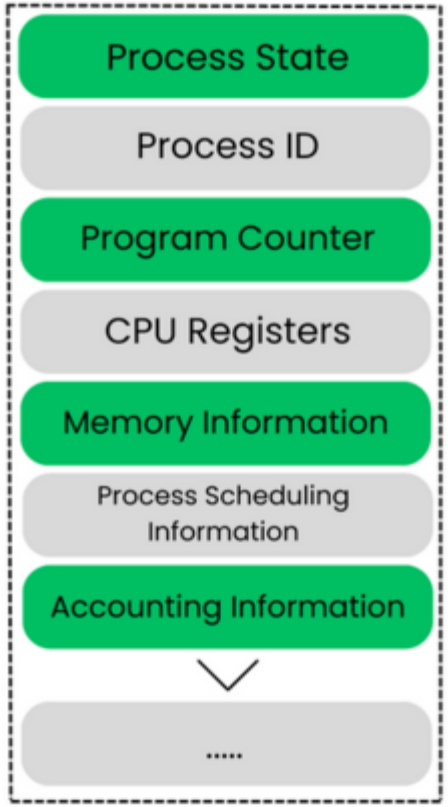


Implementation of Process Model

- To implement process model , the OS maintains a table and array of structures called the Process Table (PT) and Process Control Block (PCB).

Process Control Block

- A Process Control Block (PCB) is a data structure used by the operating system to manage information about a process.
- The PCB serves as a crucial link between the process table and the actual execution of processes.
- It contains the necessary information for the operating system to manage the process's execution, track its progress, allocate resources, and handle process-related operations.



- **Process State:** The state of the process is stored in the PCB which helps to manage the processes and schedule them. There are different states for a process which are "running," "waiting," "ready," or "terminated."
- **Process ID:** The OS assigns a unique identifier to every process as soon as it is created which is known as Process ID, this helps to distinguish between processes.
- **Program Counter:** While running processes when the context switch occurs the last instruction to be executed is stored in the program counter which helps in resuming the execution of the process from where it left off.
- **CPU Registers:** The CPU registers of the process helps to restore the state of the process so the PCB stores a copy of them.
- **Memory Information:** The information like the base address or total memory allocated to a process is stored in PCB which helps in efficient memory allocation to the processes.
- **Process Scheduling Information:** The priority of the processes or the algorithm of scheduling is stored in the PCB to help in making scheduling decisions of the OS.
- **Accounting Information:** The information such as CPU time, memory usage, etc helps the OS to monitor the performance of the process.

Process Table

- The **process table** is a data structure used by the operating system to keep track of all processes.
- It is the collection of Program control Blocks (PCBs) which contains information about each process, such as its ID (PID), current state (e.g., running, ready, waiting), CPU usage, memory allocation, and open files.
- The process table helps the operating system manage processes efficiently by tracking their progress and resource usage.
- Each entry in the table corresponds to one process, and the operating system updates it as the process moves through different states in its lifecycle.

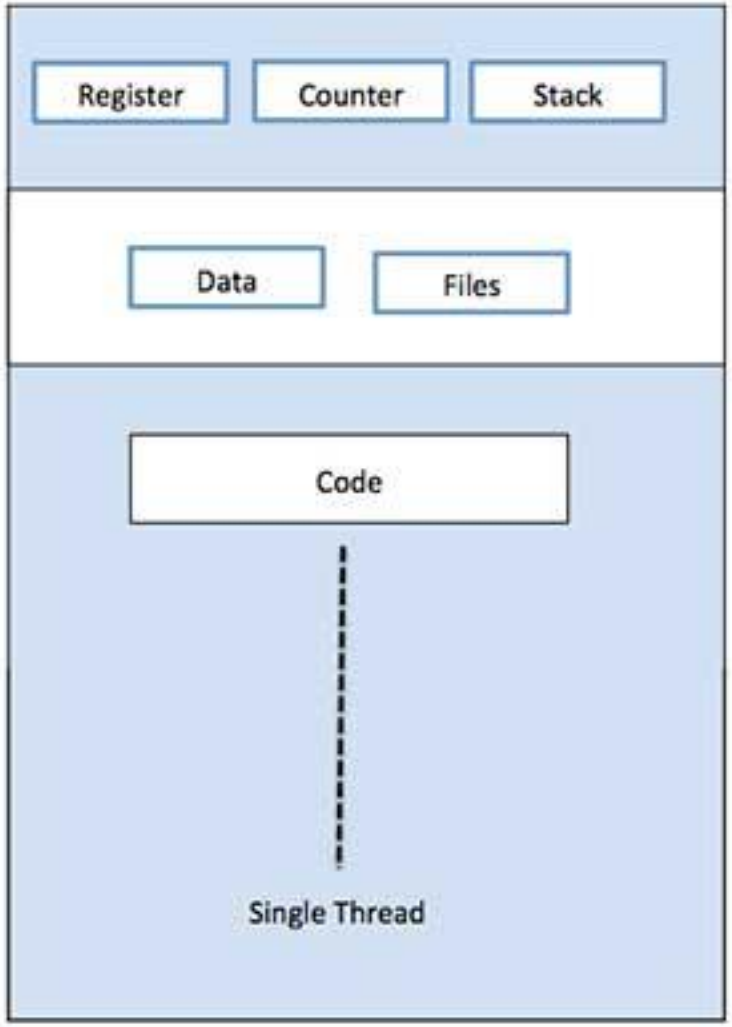
Unit 2.2 Threads

Topics to be covered

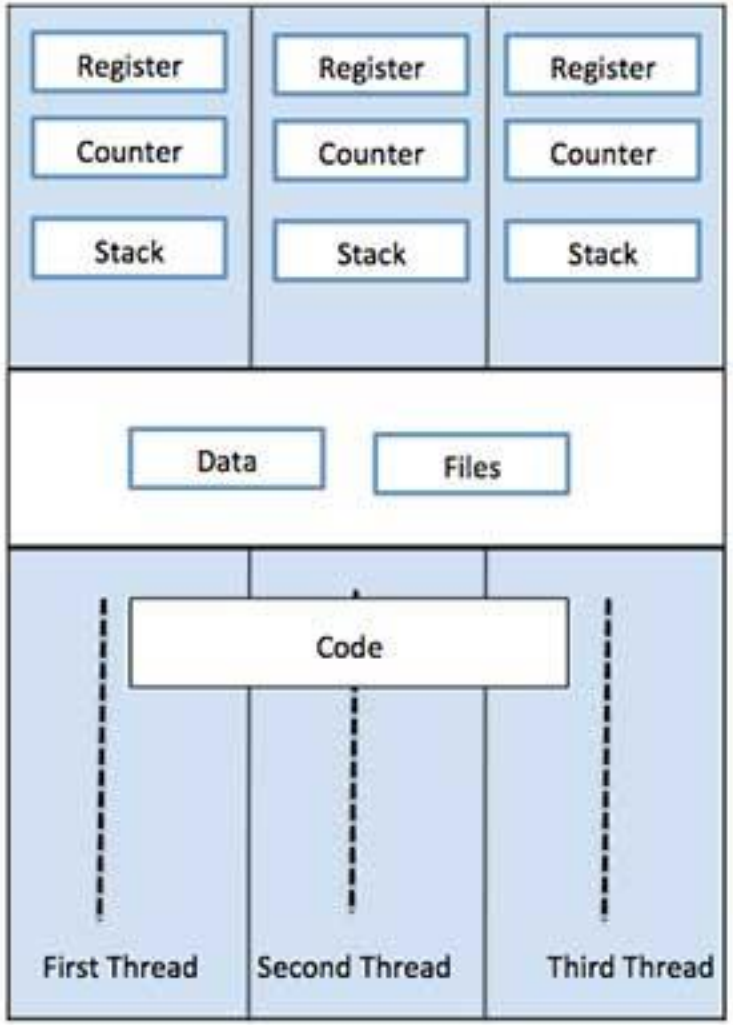
- Definition
- Thread vs Process
- Thread Usage
- User and Kernel Space Threads.

Introduction

- A thread is a single sequence stream within a process.
- Threads are also called **lightweight processes** as they possess some of the properties of processes.
- Threads provide a way to improve application performance through parallelism.
- Each thread belongs to exactly one process and no thread can exist outside a process.
- Each thread represents a separate flow of control.
- Threads have been successfully used in implementing network servers and web server.
- They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors.



Single Process P with single thread



Single Process P with three threads

Process Vs Thread

- Do it yourself

Advantages of Thread

- Threads minimize the context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- It is more economical to create and context switch threads.
- Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

Types of Thread

- Threads are implemented in following two ways –

1. **User Level Threads –** User managed threads.
2. **Kernel Level Threads –** Operating System managed threads acting on kernel, an operating system core.

User Level Threads

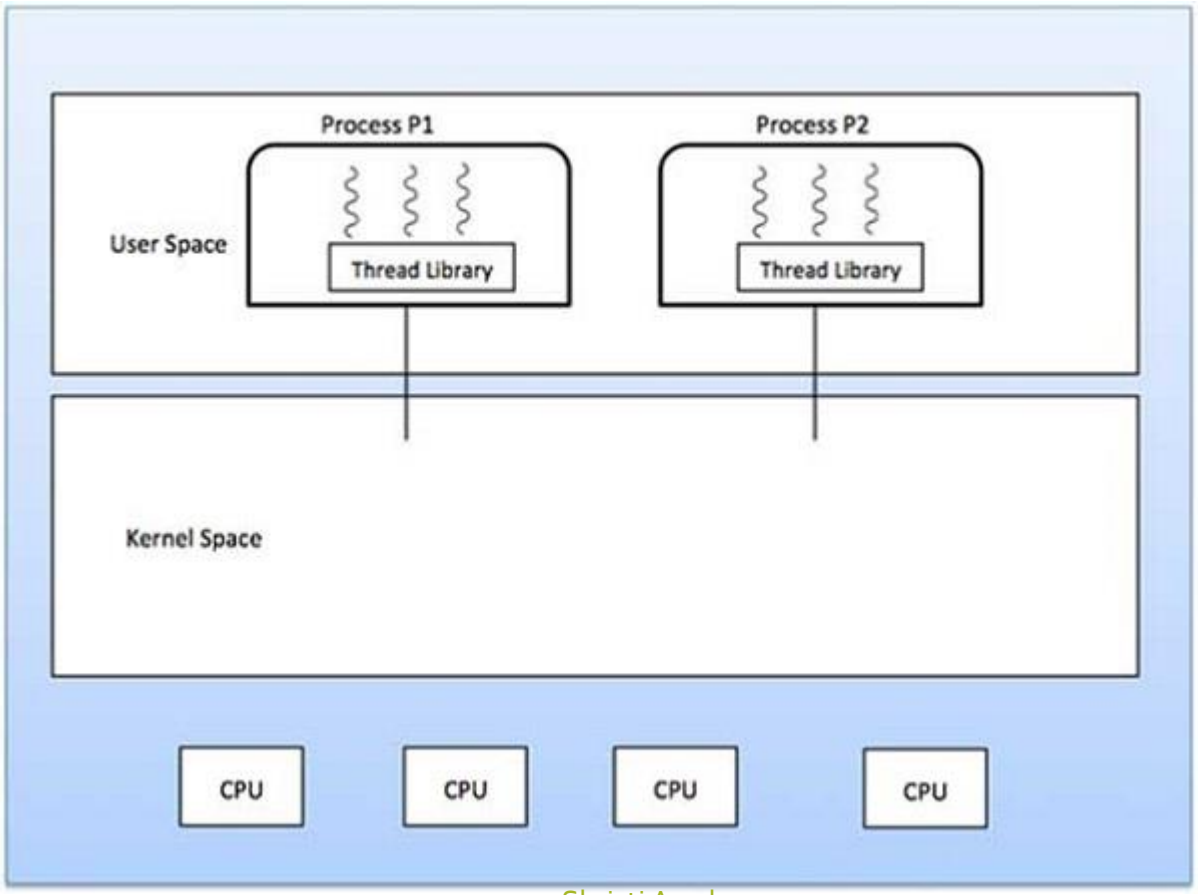
- User Level Thread is a type of thread that is not created using system calls.
- The kernel has no work in the management of user-level threads.
- User-level threads can be easily implemented by the user.
- In case when user-level threads are single-handed processes, kernel-level thread manages them.

Advantages

- Thread switching does not require Kernel mode privileges.
- User level thread can run on any operating system.
- Scheduling can be application specific in the user level thread.
- User level threads are fast to create and manage.

Disadvantages

- In a typical operating system, most system calls are blocking.
- Multithreaded application cannot take advantage of multiprocessing.



Kernel Level Threads

- A Kernel Level Thread is a type of thread that can recognize the Operating system easily.
- Kernel Level Threads has its own thread table where it keeps track of the system. The operating System Kernel helps in managing threads.
- Kernel Threads have somehow longer context switching time. Kernel helps in the management of threads.
- All of the threads within an application are supported within a single process.
- The Kernel performs thread creation, scheduling and management in Kernel space. Kernel threads are generally slower to create and manage than the user threads.

Advantages

- Kernel can simultaneously schedule multiple threads from the same process on multiple processes.
- If one thread in a process is blocked, the Kernel can schedule another thread of the same process.
- Kernel routines themselves can be multithreaded.

Disadvantages

- Kernel threads are generally slower to create and manage than the user threads.
- Transfer of control from one thread to another within the same process requires a mode switch to the Kernel.

User Level Threads Vs Kernel Level Threads

- Do it yourself

Multithreading

- Multithreading is a technique used in operating systems to improve the performance and responsiveness of computer systems.
- Multithreading allows multiple threads (i.e., lightweight processes) to share the same resources of a single process, such as the CPU, memory, and I/O devices.
- Even though the processor can only do one thing at a time, it switches between different threads from various programs so quickly that it looks like everything is happening all at once.
- Multithreading can improve the performance and efficiency of a program by utilizing the available CPU resources more effectively.
- Multithreading can enable better resource utilization. For example, in a server application, multiple threads can handle incoming client requests simultaneously, allowing the server to serve more clients concurrently.

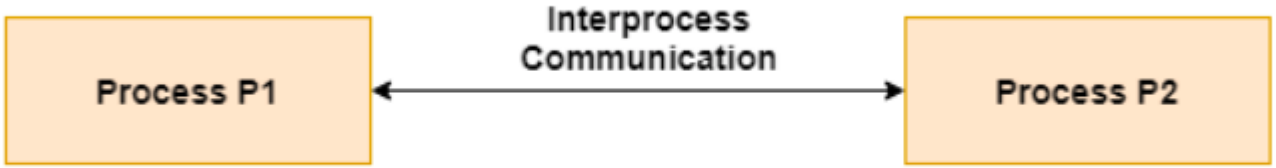
Unit 2.3 Inter Process Communication

Topics to be covered

- Introduction
- Race Condition
- Critical Section

Introduction

- **Inter-Process Communication or IPC** is a mechanism that allows processes to communicate.
- IPC lets different programs run in parallel, share data, and communicate with each other.
- In inter-process communication, messages are exchanged between two or more processes. Processes can be on the same computer or on different computers.
- important for two reasons: First, it speeds up the execution of tasks, and secondly, it ensures that the tasks run correctly and in the order that they were executed.



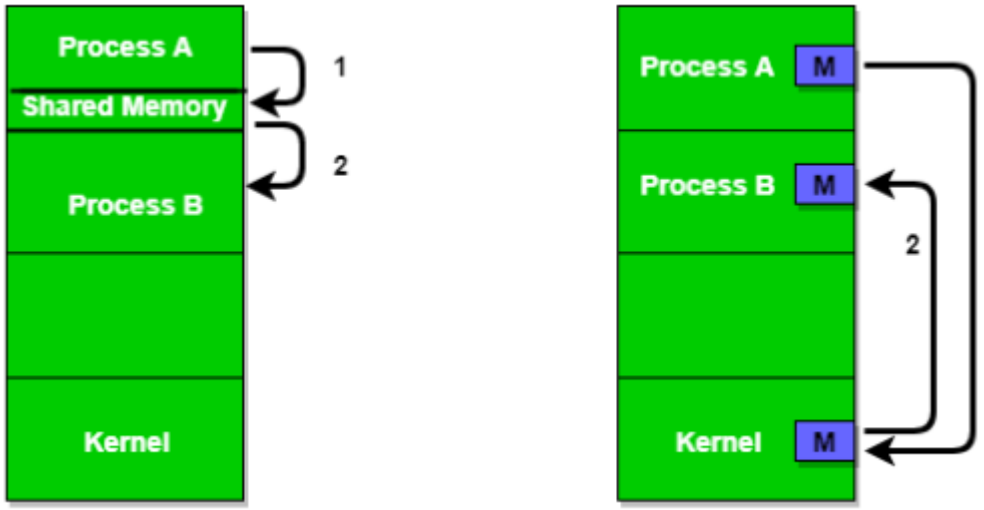
Why Inter process Communication is Necessary

- It speeds up the execution of tasks.
- It ensures that the tasks run correctly and in the order that they were executed.
- IPC is essential for the efficient operation of an operating system.
- Operating systems use IPC to exchange data with tools and components that the system uses to interact with the user, such as the keyboard, the mouse, and the graphical user interface (GUI).
- IPC also lets the system run multiple programs at the same time. For example, the system might use IPC to provide information to the windowing system about the status of a window on the screen.

- The two fundamental models of Inter Process Communication are:

1. Shared Memory: is a memory shared between all processes by two or more processes established using shared memory. This type of memory should protect each other by synchronizing access between all processes

2. Message Passing: When two or more processes participate in inter-process communication, each process sends messages to the others via Kernel. Here is an example of sending messages between two processes: – Here, the process sends a message like "M" to the OS kernel. This message is then read by Process B. A communication link is required between the two processes for successful message exchange.



Race Condition

- A race condition is a situation that may occur inside a critical section.
- This happens when the result of multiple thread execution in a critical section differs according to the order in which the threads execute.
- Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction.
- In computer memory or storage, a race condition may occur if commands to read and write a large amount of data are received at almost the same instant, and the machine attempts to overwrite some or all of the old data while that old data is still being read. The result may be one or more of the following:
  - the computer crashes or identifies an illegal operation of the program
  - errors reading the old data
  - errors writing the new data

- Two categories that define the impact of the race condition on a system are referred to as critical and noncritical:
- A **critical** race condition will cause the end state of the device, system or program to change. For example, if flipping two light switches connected to a common light at the same time blows the circuit, it is considered a critical race condition. In software, a critical race condition is when a situation results in a bug with unpredictable or undefined behavior.
- A **noncritical** race condition does not directly affect the end state of the system, device or program. In the light example, if the light is off and flipping both switches simultaneously turns the light on and has the same effect as flipping one switch, then it is a noncritical race condition. In software, a noncritical race condition does not result in a bug.

Examples

- Imagine two people sending print jobs at the same time. If the printer isn't managed properly, the print jobs could get mixed up, with pages from one person's document being printed in the middle of another's.
- Consider a situation where two employees are working on the same document in a company's shared folder. Employee A opens the document and starts editing it, but before they can save their changes, Employee B opens the same document and starts making changes. When Employee A tries to save their changes, he is unable to change because the document has been updated by Employee B, and their changes are lost. To avoid race conditions in this, the company may implement a version control system or a check-out/check-in mechanism that allows employees to work on different copies of the document and merge their changes later.

Critical Section

- A **critical section** is a part of a program where shared resources like memory or files are accessed by multiple processes or threads.
- To avoid issues like data inconsistency or race conditions, synchronization techniques ensure that only one process or thread uses the critical section at a time.
- It is essential for the operating system to provide mechanisms like locks and semaphores to ensure proper synchronization and mutual exclusion in the critical section.

- If we could arrange matters such that no two processes were ever in their critical regions at the same time we could avoid races. We need four conditions to hold to have a good solution:
  - No two processes may be simultaneously inside their critical regions.
  - No assumptions may be made about speeds or the number of CPUs.
  - No process running outside its critical region may block other processes.
  - No process should have to wait forever to enter its critical region.

Solution to the Critical Section Problem

- **Mutual Exclusion:** Mutual exclusion implies that only one process can be inside the critical section at any time. If any other processes require the critical section, they must wait until it is free.
- **Progress:** Progress means that if a process is not using the critical section, then it should not stop any other process from accessing it. In other words, any process can enter a critical section if it is free.
- **Bounded Waiting:** Bounded waiting means that each process must have a limited waiting time. Itt should not wait endlessly to access the critical section.

Semaphore

- A semaphore is a signaling mechanism and a thread that is waiting on a semaphore can be signaled by another thread.
- This is different than a mutex as the mutex can be signaled only by the thread that called the wait function.
- A semaphore uses two atomic operations, wait and signal for process synchronization.
- The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed.

Unit 2.4 Implementing Mutual Exclusion

Topics to be covered

- Mutual Exclusion with Busy Waiting (Disabling Interrupts, Lock Variables, Strict Alteration, Peterson's Solution, Test and Set Lock),
- Sleep and Wakeup,
- Semaphore,
- Monitors,
- Message Passing

Mutual Exclusion

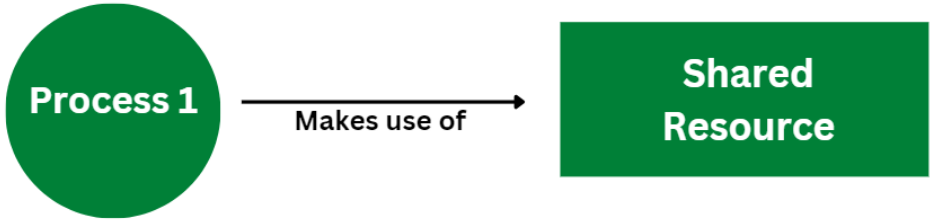
- Mutual Exclusion is a property of process synchronization that states that "no two processes can exist in the critical section at any given point of time".
- Mutual exclusion methods are used in concurrent programming to avoid the simultaneous use of a common resource, such as a global variable, by pieces of computer code called critical sections.
- The requirement of mutual exclusion is that when process P1 is accessing a shared resource R1, another process should not be able to access resource R1 until process P1 has finished its operation with resource R1.

Busy waiting

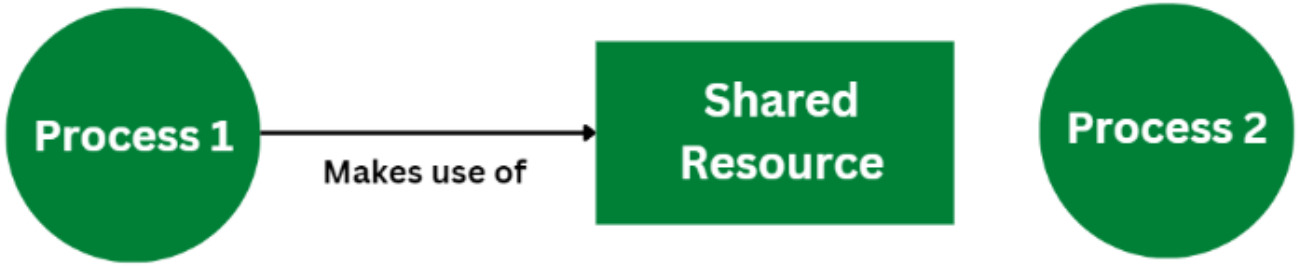
- Busy Waiting is defined as a process synchronization technique where the process waits and continuously keeps on checking for the condition to be satisfied before going ahead with its execution.

Demonstration of Busy Waiting

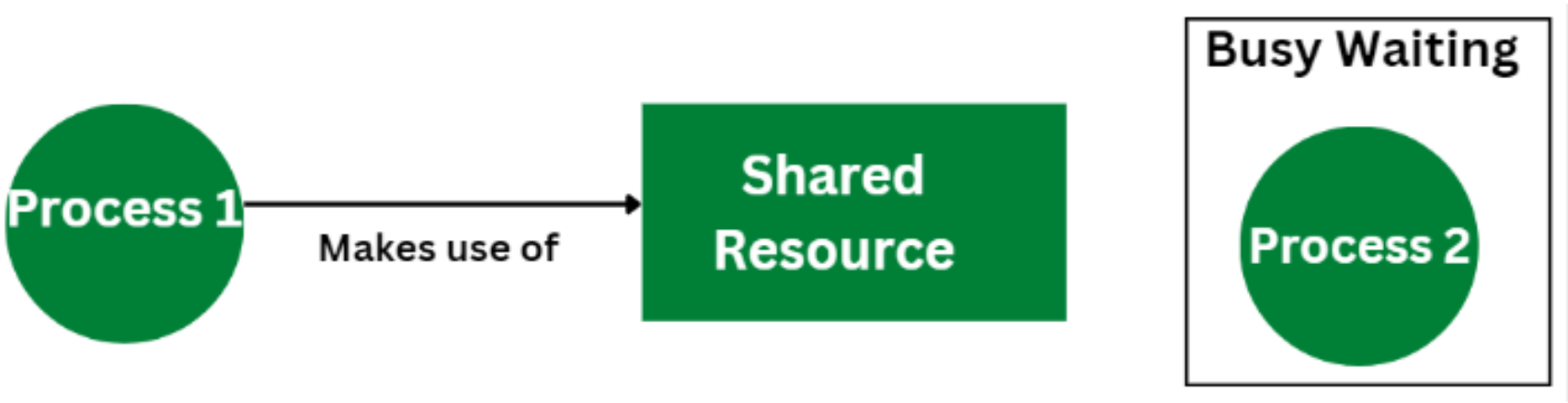
Step 1: Process 1 makes use of a shared resource



Step 2: Process 2 needs the shared resource that is already being used by process 1



Step 3: Process 2 enters into busy waiting state until its get access to the shared resource



Limitations of Busy Waiting

- Busy waiting keeps CPU busy at all the time until it's condition gets satisfied.
- The synchronization mechanism that makes use of busy waiting suffers from the problem of priority inversion.
- In critical section the low priority processes gets executed.
- The system remains idle when process is in busy waiting state.
- Busy waiting consumes more power.

Proposals for Achieving Mutual Exclusion

1. Disabling Interrupts: On a single-processor system, the simplest solution is to have each process disable all interrupts just after entering its critical region and re-enable them just before leaving it. With interrupts disabled, no clock interrupts can occur. The CPU is only switched from process to process as a result of clock or other interrupts, after all, and with interrupts turned off the CPU will not be switched to another process. Thus, once a process has disabled interrupts, it can examine and update the shared memory without fear that any other process will intervene. This approach is generally unattractive because it is unwise to give user processes the power to turn off interrupts. Suppose that one of them did it, and never turned them on again? That could be the end of the system. Furthermore, if the system is a multiprocessor (with two or possibly more CPUs) disabling interrupts affects only the CPU that executed the disable instruction. The other ones will continue running and can access the shared memory.

2. Lock Variables: As a second attempt, let us look for a software solution. Consider having a single, shared (lock) variable, initially 0. When a process wants to enter its critical region, it first tests the lock. If the lock is 0, the process sets it to 1 and enters the critical region.

- If the lock is already 1, the process just waits until becomes 0. Thus, a 0 means that no process is in its critical region, and a 1 means that some process is in its critical region.
- Unfortunately, this idea contains exactly the same fatal flaw that we saw in the spooler directory. Suppose that one process reads the lock and sees that it is 0.
- Before it can set the lock to 1, another process is scheduled, runs, and sets the lock to 1. When the first process runs again, it will also set the lock to 1, and two processes will be in their critical regions at the same time.

3. Strict Alternation: The integer variable turn, initially 0, keeps track of whose turn it is to enter the critical region and examine or update the shared memory. Initially, process 0 inspects turn, finds it to be 0, and enters its critical region. Process 1 also finds it to be 0 and therefore sits in a tight loop continually testing turn to see when it becomes 1.

When process 0 leaves the critical region, it sets turn to 1, to allow process 1 to enter its critical region. Suppose that process 1 finishes its critical region quickly, so that both processes are in their noncritical regions, with turn set to 0.

Now process 0 executes its whole loop quickly, exiting its critical region and setting turn to 1. At this point turn is 1 and both processes are executing in their noncritical regions.

4. Peterson's Algorithm: Before using the shared variables (i.e., before entering its critical region), each process calls enter_region with its own process number, 0 or 1, as parameter. This call will cause it to wait, if need be, until it is safe to enter. After it has finished with the shared variables, the process calls leave_region to indicate that it is done and to allow the other process to enter, if it so desires.

Initially neither process is in its critical region. Now process 0 calls enter_region. It indicates its interest by setting its array element and sets turn to 0.

Since process 1 is not interested, enter region returns immediately. If process 1 now makes a call to enter_region, it will hang there until interested [0] goes to FALSE, an event that only happens when process 0 calls leave_region to exit the critical region.

- **Advantages**: preserves all condition of ME.
- **Disadvantages**: Difficulty to program for n-process system and less efficient.

5. The TSL Instruction

```
enter_region:
TSL REGISTER.LOCK: copy lock to register and set lock to 1
CMP REGISTER, #0     was lock zero?
JNE enter_region     if it was nonzero, lock was set, so loop
RET                  return to caller; critical region entered
leave_region:
MOVE LOCK, #0        store a 0 in lock
RET                  return to caller
```

To use the TSL instruction, we will use a shared variable, *lock*, to coordinate access to shared memory. When *lock* is 0, any process may set it to 1 using the TSL instruction and then read or write the shared memory. When it is done, the process sets *lock* back to 0 using an ordinary move instruction.

- **Advantages**: Preserves all conditions, easier programming task and improve system efficiency.
- **Problem:** Difficulty in hardware design.

Alternative of Busy waiting

- Both Peterson's algorithm and the solution using TSL are correct, but both have the defect of requiring busy waiting.
- When a process want to enter its CR, it checks to see if the entry is allowed, if it is not, the process just sits in a tight loop waiting until it is, This cause waste of CPU Timer for noting

1. Sleep and Wakeup

- Sleep is a system call that causes the caller to block, that is, be suspended until another process wakes it up. The wakeup call has one parameter, the process to be awakened.
- Alternatively, both sleep and wakeup each have one parameter, a memory address used to match up sleeps with wakeups
- **Sleep():** When a process calls this system call, it gets blocked, managing it suspends its execution until another process wakes it up.
- **Wakeup():** When another process calls this system call, it triggers the awakening of a process that was previously put to sleep.
- .

Semaphore

- Semaphore is a simply a variable. This variable is used to solve critical section problem and to achieve process synchronization in the multiprocessing environment.
- Semaphores can be of two types, binary semaphores and counting semaphores.
- Counting semaphore can take non-negative integer values and Binary semaphore can take the value 0 & 1 only.
- **A semaphore can only be accessed using the following operations:**
- **wait ()** :- called when a process wants access to a resource
- **signal ()**:- called when a process is done using a resource
- In a counting semaphore, when a thread needs to enter the critical area, semaphores keep track of a counter and decrease it.
- The running thread becomes immobilized if the counter decreases, signifying that the critical portion has become in use.

Monitors

- Codes in semaphores are like assembly language code.
- To make easier to write correct program, Hoare and Brinch Hasen proposed a higher-level synchronization primitive called monitor.
- A monitor is a collection of procedures, variables and data structure that all grouped together in a special kind of module or package.
- Process may call the procedures in a monitor whenever they want to, but they cannot directly access the monitor's internal data structures form procedures declared outside the monitor.
- Monitor have an important property that makes them useful for achieving ME: only one process can be active in a monitor at any instant.

- Monitors are a programming language construct, so the complier knows they are special and can handle calls to monitor procedure, the first few instruction of the procedure will check to see if any other process is currently active within the monitor.
- If so, the calling process will be suspended until the other process has left the monitor.
- If no other process is using the monitor, the calling process may enter. So, no two process will executing their CR at the same time.

Message Passing

- With the trend of distributed OS, many OS are used to communicate through internet, intranet and remote data processing etc.
- Message passing method of inter-process communication uses two primitive send and receive.

Unit 2.5 Classical IPC problems

Topic to be covered

- Producer Consumer,
- Sleeping Barber, and
- Dining Philosopher Problem

Introduction

- Classical IPC problems are used for process synchronization
  1. Producer- Consumer Problem (Buffer Bounded Problem)
  2. Sleeping Barber Problem
  3. Dining Philosopher Problem

Producer-Consumer(Buffer Bounded Problem)

- In this problem two process share a common, fixed-size buffer.
- The producer puts information into the buffer and the consumer takes it out.
- Trouble arises when the producer wants to put a new item in the buffer, but it its already full.
- The solution is the producer should go to sleep, and to be awakened when the consumer has removed one or more items.
- Similarly, if the consumer wants to remove an item from the buffer and sees that the buffer is empty, it goes to sleep until the producer puts something in th buffer and wakes it up.

- To keep the track of the no.of items in the buffer we will need a variable count.
- If the maximum no.of items the buffer can hold is N, then producers code will first test to see if count is N.
- If it is, then the producer code will first test to see count is N.
- If it is then the producer will go to sleep otherwise producer will add an item and increments count.
- The consumer's code is also similar: first it test count if it is 0. If it is then goes to sleep.
- If it is nonzero, it removes an item and decrements count.

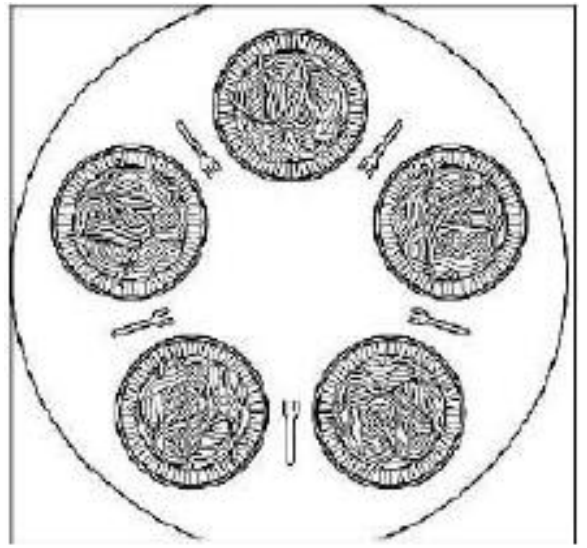
Sleeping Barber Problem

- In this problem there is a barber shop with one barber, one barber chair and n chairs waiting for customers if there are any to sit on the chair.
- If there is no customer, then barber sleeps in his own chair.
- When a customer arrives, he has to wake up the barber.
- If there are many customers and the barber is cutting a customer's hair, then remaining customers either wait if there are empty chairs in waiting room or they leave if no chairs are empty.

- The solution to this problem includes three semaphores.
- First is for the customer which counts ,o.of customers present in the waiting room.
- Second barber , 0 or 1 is used to tell whether the barber is idle or working and third mutex is used to provide the mutual exclusion required for process to execute.
- When customer arrives, he requires the mutex for entering critical region, if another customer enters thereafter, the second one will not be able to anything until the first one has released the mutex.
- If the chair is available then the customer sits int the waiting room and increments the variable and also increase the customers semaphore this wakes up the barber if he is sleeping.
- At this point, customer and barber both are awake and the barber is ready to give that person a haircut.

Dining Philosopher Problem

- Five philosophers are seated around a circular table. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork.
- The life of philosopher consists of alternative period of eating and thinking . When philosopher gets hungry, she tries to acquire her left and right forks; she eats for a while, then puts down the forks and continues to think.



Solution for dining philosopher problem

1. Problem: The problem is can we write a program for each philosopher that does what is supposed to do and never gets stuck?

Solution: When the philosopher is hungry it picks up a fork and wait for another fork, when get it eats for a while and put both forks back to the table.

2. Problem: Suppose that all the philosopher take their left fork simultaneously none will be able to take their right fork and therefore will be a deadlock.

Solution: After taking left fork, the program checks for right fork, if it is not available, the philosopher puts down the left fork one, waits for some time and then repeats the whole process.

3. Problem: All philosopher could start the process of picking left fork simultaneously and so on. In this case starvation may occur.

Solution: Using binary semaphore mutex before starting to acquire a fork she would do down on mutex. After replacing the fork she would do up on the mutex.

Unit 2.6 Process Scheduling

Topics to be covered

- Batch System Scheduling (First-Come First-Served, Shortest Job First, Shortest Remaining Time Next),
- Interactive System Scheduling (Round-Robin Scheduling, Priority Scheduling, Multiple Queues),
- Overview of Real Time System Scheduling (*No need to discuss any real time system scheduling algorithm)*

What is process scheduling?

- Process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process based on a particular strategy.
- Throughout its lifetime, a process moves between various scheduling queues, such as the ready queue, waiting queue, or devices queue.
- Process Scheduling is responsible for selecting a processor process based on a scheduling method as well as removing a processor process.
- Process scheduling makes use of a variety of scheduling queues.

Process Scheduling Queues

- The OS maintains all Process Control Blocks (PCBs) in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.
- The Operating System maintains the following important process scheduling queues
- **Job queue** – This queue keeps all the processes in the system.
- **Ready queue** – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- **Device queues** – The processes which are blocked due to unavailability of an I/O device constitute this queue.

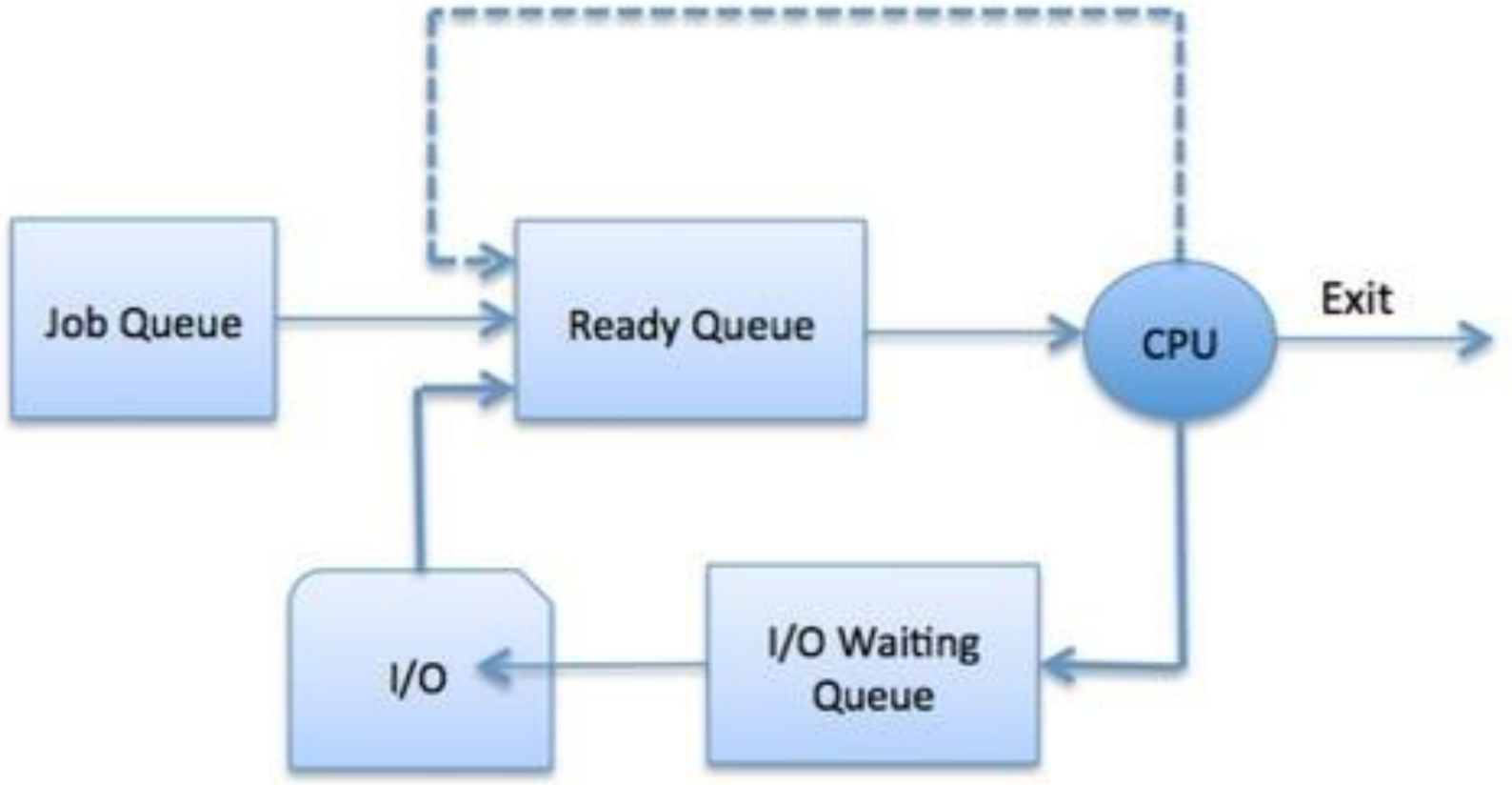


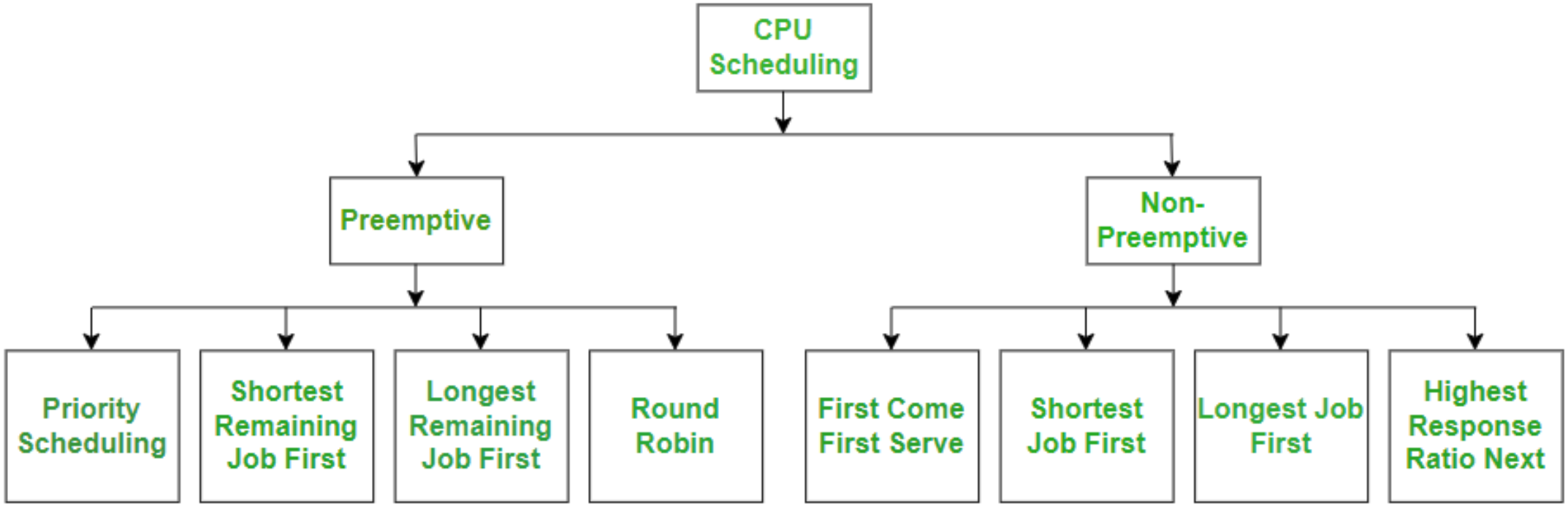
Figure: Process Scheduling

What is the Need for a CPU Scheduling Algorithm?

- **CPU scheduling** is the process of deciding which process will own the CPU to use while another process is suspended.
- The main function of CPU scheduling is to ensure that whenever the CPU remains idle, the OS has at least selected one of the processes available in the ready-to-use line.
- In Multiprogramming, if the long-term scheduler selects multiple I/O binding processes then most of the time, the CPU remains idle.
- The function of an effective program is to improve resource utilization.

Terminologies Used in CPU Scheduling

- **Arrival Time:** The time at which the process arrives in the ready queue.
- **Completion Time:** The time at which the process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.
  Turn Around Time = Completion Time – Arrival Time
- **Waiting Time(W.T):** Time Difference between turn around time and burst time.
  Waiting Time = Turn Around Time – Burst Time


CPU Scheduling
Preemptive
Non-Preemptive
Priority Scheduling
Shortest Remaining Job First
Longest Remaining Job First
Round Robin
First Come First Serve
Shortest Job First
Longest Job First
Highest Response Ratio Next

Categories of Scheduling

- **Non-preemptive:** Here the resource cant be taken from a process until the process completes execution. The switching of resources occurs when the running process terminates and moves to a waiting state. Algorithms based on non-preemptive scheduling are: First Come First Serve, Shortest Job First (SJF basically non preemptive) and Priority (nonpreemptive version) , etc.
- **Preemptive:** Here the OS allocates the resources to a process for a fixed amount of time. During resource allocation, the process switches from running state to ready state or from waiting state to ready state. This switching occurs as the CPU may give priority to other processes and replace the process with higher priority with the running process. . Algorithms based on preemptive scheduling are Round Robin (RR) , Shortest Remaining Time First (SRTF) , Priority (preemptive version) , etc.

Advantages of Preemptive Scheduling

- Because a process may not monopolize the processor, it is a more reliable method and does not cause denial of service attack.
- Each preemption occurrence prevents the completion of ongoing tasks.
- The average response time is improved.
- Utilizing this method in a multi-programming environment is more advantageous.
- Most of the modern operating systems (Window, Linux and macOS) implement Preemptive Scheduling.

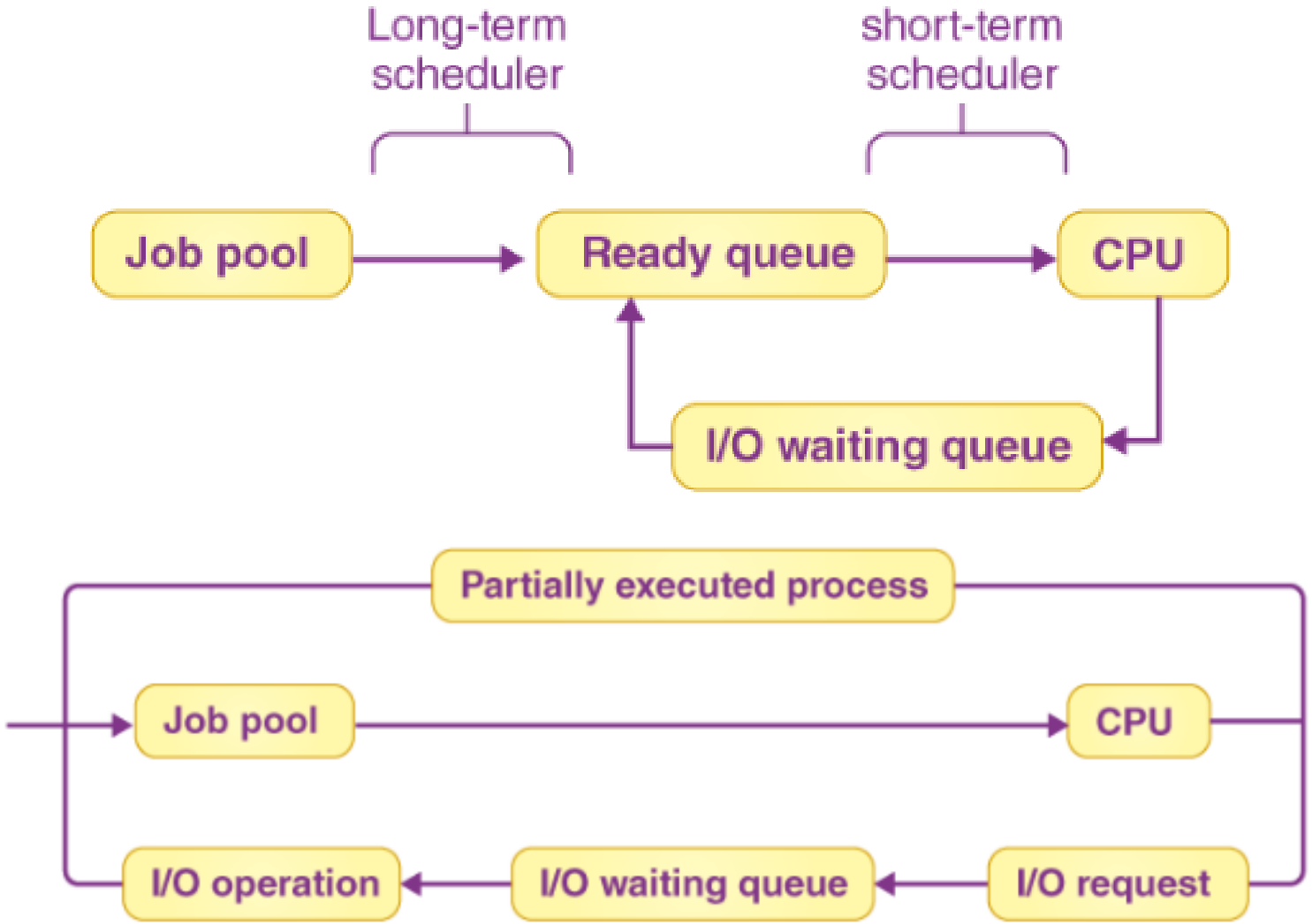
Disadvantages of Preemptive Scheduling

- More Complex to implement in Operating Systems.
- Suspending the running process, change the context, and dispatch the new incoming process all take more time.
- Might cause starvation : A low-priority process might be preempted again and again if multiple high-priority processes arrive.
- Causes Concurrency Problems as processes can be stopped when they were accessing shared memory (or variables) or resources.

- **Advantages of Non-Preemptive Scheduling**
- It is easy to implement in an operating system. It was used in Windows 3.11 and early macOS.
- It has a minimal scheduling burden.
- Less computational resources are used.
- **Disadvantages of Non-Preemptive Scheduling**
- It is open to denial of service attack.
- A malicious process can take CPU forever.
- Since we cannot implement round robin, the average response time becomes less.

Schedulers

- Schedulers are special system software which handle process scheduling in various ways.
- Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types:
  - Long-Term Scheduler
  - Short-Term Scheduler
  - Medium-Term Scheduler


Long-term scheduler
short-term scheduler
Job pool
Ready queue
CPU
I/O waiting queue
Partially executed process
Job pool
CPU
I/O operation
I/O waiting queue
I/O request

Long-Term Scheduler or Job Scheduler

- The job scheduler is another name for Long-Term scheduler. It selects processes from the pool (or the secondary memory) and then maintains them in the primary memory's ready queue.
- The Multiprogramming degree is mostly controlled by the Long-Term Scheduler. The goal of the Long-Term scheduler is to select the best mix of IO and CPU bound processes from the pool of jobs.
- If the job scheduler selects more IO bound processes, all of the jobs may become stuck, the CPU will be idle for the majority of the time, and multiprogramming will be reduced as a result. Hence, the Long-Term scheduler's job is crucial and could have a Long-Term impact on the system.

Short-Term Scheduler or CPU Scheduler

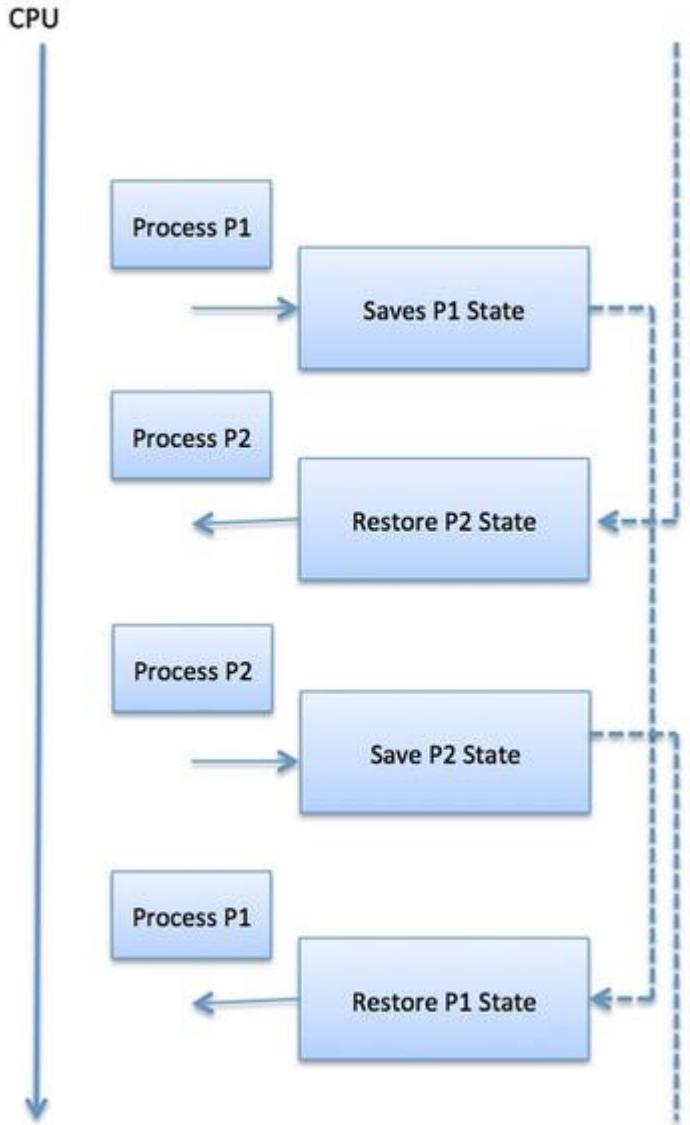
- It chooses one job from the ready queue and then sends it to the CPU for processing.
- To determine which work will be dispatched for execution, a scheduling method is utilized.
- The Short-Term scheduler's task can be essential in the sense that if it chooses a job with a long CPU burst time, all subsequent jobs will have to wait in a ready queue for a long period. This is known as hunger, and it can occur if the Short-Term scheduler makes a mistake when selecting the work.

Medium-Term Scheduler

- The switched-out processes are handled by the Medium-Term scheduler. If the running state processes require some IO time to complete, the state must be changed from running to waiting.
- This is accomplished using a Medium-Term scheduler. It stops the process from executing in order to make space for other processes. Swapped out processes are examples of this, and the operation is known as swapping. The Medium-Term scheduler here is in charge of stopping and starting processes.
- The degree of multiprogramming is reduced. To have a great blend of operations in the ready queue, swapping is required.

Context Switching

- A context switching is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time. Using this technique, a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking operating system features.
- When the scheduler switches the CPU from executing one process to execute another, the state from the current running process is stored into the process control block. After this, the state for the process to run next is loaded from its own PCB and used to set the PC, registers, etc. At that point, the second process can start executing.


CPU
Process P1
Saves P1 State
Process P2
Restore P2 State
Process P2
Save P2 State
Process P1
Restore P1 State

Scheduling Algorithm

- There are six popular process scheduling algorithms which we are going to discuss in this chapter –
  - First-Come, First-Served (FCFS) Scheduling
  - Shortest-Job-First (SJF) Scheduling
  - Priority Scheduling
  - Shortest Remaining Time
  - Round Robin(RR) Scheduling
  - Multiple-Level Queues Scheduling
- These algorithms are either **non-preemptive or preemptive**. Non-preemptive algorithms are designed so that once a process enters the running state, it cannot be preempted until it completes its allotted time, whereas the preemptive scheduling is based on priority where a scheduler may preempt a low priority running process anytime when a high priority process enters into a ready state.

First Come First Serve (FCFS)

- Jobs are executed on first come, first serve basis.
- The job that requests execution first gets the CPU allocated to it, then the second, and so on.
- It is a non-preemptive scheduling.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

Advantages of FCFS scheduling algorithm

- The fact that it is simple to implement means it can easily be integrated into a pre-existing system.
- It is especially useful when the processes have a large burst time since there is no need for context switching.
- The absence of low or high-priority preferences makes it fairer.
- Every process gets its chance to execute.

Disadvantages of the FCFS scheduling algorithm

- Since its first come basis, small processes with a very short execution time, have to wait their turn.
- There is a high wait and turnaround time for this scheduling algorithm in OS.
- All in all, it leads to inefficient utilization of the CPU.

How Does FCFS Work?

- The mechanics of FCFS are straightforward:
- **Arrival:** Processes enter the system and are placed in a queue in the order they arrive.
- **Execution:** The CPU takes the first process from the front of the queue, executes it until it is complete, and then removes it from the queue.
- **Repeat:** The CPU takes the next process in the queue and repeats the execution process.
- This continues until there are no more processes left in the queue.

Example of FCFS

- **Scenario 1: Processes with Same Arrival Time**

Process	Arrival Time	Burst Time
p1	0	5
p2	0	3
p3	0	8

1. **P1** will start first and run for 5 units of time (from 0 to 5).
2. **P2** will start next and run for 3 units of time (from 5 to 8).
3. **P3** will run last, executing for 8 units (from 8 to 16).

- Now, let's calculate average waiting time and turn around time:
  - Turnaround Time = Completion Time - Arrival Time
  - Waiting Time = Turnaround Time - Burst Time

Processes	AT	BT	CT	TAT	WT
P1	0	5	5	5-0 = 5	5-5 = 0
P2	0	3	8	8-0 = 8	8-3 = 5
P3	0	8	16	16-0 = 16	16-8 = 8

- **Average Turn around time** = 9.67
- **Average waiting time** = 4.33

Implementation

- **Input** : Processes along with their burst time (bt).
  Output : Waiting time, turnaround time for all processes.
- As first process that comes need not to wait so waiting time for process 1 will be 0 i.e. wt[0] = 0.
- Find **waiting time** for all other processes i.e. for process i:
  wt[i] = bt[i-1] + wt[i-1] .
- Find **turnaround time** = waiting_time + burst_time for all processes.
- Find **average waiting time** = total_waiting_time / no_of_processes.
- Similarly, find **average turnaround time** = total_turn_around_time / no_of_processes.

- **Scenario 2: Processes with Different Arrival Times**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	2 ms
P2	3 ms	0 ms
P3	4 ms	4 ms

1. **P2** arrives at time 0 and runs for 3 units, so its completion time is:
   Completion Time of P2=0+3=3
2. **P1** arrives at time 2 but has to wait for **P2** to finish. **P1** starts at time 3 and runs for 5 units. Its completion time is:
   Completion Time of P1=3+5=8
3. **P3** arrives at time 4 but has to wait for **P1** to finish. **P3** starts at time 8 and runs for 4 units. Its completion time is:
   Completion Time of P3=8+4=12

- Now, lets calculate average waiting time and turn around time:

Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P2	3 ms	3 ms	0 ms
P1	8 ms	6 ms	1 ms
P3	12 ms	8 ms	4 ms

- Average Turnaround time = 5.67
- Average waiting time = 1.67

Implementation

1- Input the processes along with their burst time(bt) and arrival time(at)

2- Find waiting time for all other processes i.e. for a given process i:

wt[i] = (bt[0] + bt[1] +..... bt[i-1]) - at[i]

3- Now find turn around time = waiting_time + burst_time for all processes

4- Average waiting time = total_waiting_time / no_of_processes

5- Average turn around time = total_turn_around_time / no_of_processes

Shortest-Job-First (SJF) Scheduling

- The idea is to quickly get done with jobs that have short/ lowest CPU burst time, making CPU available for other, longer jobs/ processes.
- It is easier to implement the SJF algorithm in Batch OS.
- Best approach to minimize waiting time.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processer should know in advance how much time process will take.
- It is a Greedy Algorithm.

Advantages of SJF scheduling algorithm

- It minimizes the average waiting time and turnaround time.
- Beneficial in long-term scheduling.
- It is better than the FCFS scheduling algorithm.
- Useful for batch processes.

Disadvantages of the SJF scheduling algorithm

- As mentioned, if short time jobs keep on coming, it may lead to starvation for longer jobs.
- It is dependent upon burst time, but it is not always possible to know the burst time beforehand.
- Does not work for interactive systems.

Example of Non Pre-emptive Shortest Job First CPU Scheduling Algorithm

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	8 ms	2 ms
P3	3 ms	4 ms

1. **Time 0-6 (P1)**: P1 runs for 6 ms (total time left: 0 ms)
2. **Time 6-9 (P3)**: P3 runs for 3 ms (total time left: 0 ms)
3. **Time 9-17 (P2)**: P2 runs for 8 ms (total time left: 0 ms)

- Now, lets calculate average waiting time and turn around time:

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	6	6-0 = 6	6-6 = 0
P2	2	8	17	17-2 = 15	15-8 = 7
P3	4	3	9	9-4 = 5	5-3 = 2

- ***Average Turn around time*** *= (6 + 15 + 5)/3 = 8.6 ms*
- ***Average waiting time*** *= (2 + 0 + 7)/3 = 9/3 = 3 ms*

Shortest Remaining Time

- The SRTF scheduling algorithm is the preemptive version of the SJF scheduling algorithm in OS.
- This calls for the job with the shortest burst time remaining to be executed first, and it keeps preempting jobs on the basis of burst time remaining in ascending order.
- It requires the least burst time to be executed first, but if another process arrives that has an even lesser burst time, then the former process will get preempted for the latter.
- The flow of execution is- a process is executed for some specific unit of time and then the scheduler checks if any new processes with even shorter burst times have arrived.

Advantages of SRTF Scheduling Algorithm

- More efficient than SJF since it's the preemptive version of SJF.
- Efficient scheduling for batch processes.
- The average waiting time is lower in comparison to many other scheduling algorithms in OS.

Disadvantages of SRTF Scheduling Algorithm

- Longer processes may starve if short jobs keep getting the first shot.
- Can’t be implemented in interactive systems.
- The context switch happens too many times, leading to a rise in the overall completion time.
- The remaining burst time might not always be apparent before the execution.

Scenario 1: Processes with Same Arrival Time

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	8 ms	0 ms
P3	5 ms	0 ms

1. **Time 0-5 (P3)**: P3 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left.
2. **Time 5-11 (P1)**: P1 runs for 6 ms (total time left: 0 ms) as it has shortest remaining time left.
3. **Time 11-19 (P2)**: P2 runs for 8 ms (total time left: 0 ms) as it has shortest remaining time left.

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	11	11-0 = 11	11-6 = 5
P2	0	8	19	19-0 = 19	19-8 = 11
P3	0	5	5	5-0 = 5	5-5 = 0

- ***Average Turn around time*** *= (11 + 19 + 5)/3 = 11.6 ms*
- ***Average waiting time*** *= (5 + 0 + 11)/3 = 16/3 = 5.33 ms*

Scenario 2: Processes with Different Arrival Times

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	3 ms	1 ms
P3	7 ms	2 ms

1. **Time 0-1 (P1)**: P1 runs for 1 ms (total time left: 5 ms) as it has shortest remaining time left.
2. **Time 1-4 (P2)**: P2 runs for 3 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P2.
3. **Time 4-9 (P1)**: P1 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P3.
4. **Time 9-16 (P3)**: P3 runs for 7 ms (total time left: 0 ms) as it has shortest remaining time left.

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	9	9-0 = 9	9-6 = 3
P2	1	3	4	4-1 = 3	3-3 = 0
P3	2	7	16	16-2 = 14	14-7 = 7

- ***Average Turn around time*** *= (9 + 14 + 3)/3 = 8.6 ms*
- ***Average waiting time*** *= (3 + 0 + 7)/3 = 10/3 = 3.33 ms*

Priority Scheduling

- The job with the highest priority gets executed first, followed by the second prioritized jobs, and so on.
- If a job with higher priority than the one running currently comes on, the CPU preempts the current job in favor of the one with higher priority.
- But for other purposes, it follows a non-preemptive scheduling approach.
- In between two jobs with the same priority, the FCFS process decides which jobs get executed first.
- The priority of a process can be set depending on multiple factors like memory requirements, required CPU time, etc.

Advantages of Priority Scheduling Algorithm

- This process is simpler than most other scheduling algorithms in OS.
- Priorities help in sorting the incoming processes.
- Works well for static and dynamic environments.

Disadvantages of Priority Scheduling Algorithm

- It may lead to the starvation problem in jobs with low priority.
- The average turnaround and waiting time might be higher in comparison to other CPU scheduling algorithms.

Priority Scheduling can be implemented in two ways:

- Non-Preemptive Priority Scheduling
- Preemptive Priority Scheduling

Preemptive Priority Scheduling

- In **Preemptive Priority Scheduling**, the CPU can be taken away from the currently running process if a new process with a higher priority arrives.
- Ex: A low-priority process is running, and a high-priority process arrives; the CPU immediately switches to the high-priority process.
- **Example of Preemptive Priority Scheduling (Same Arrival Time)**
- *Note: Higher number represents higher priority.*

Process	Arrival Time	Burst Time	Priority
P1	0	7	2
P2	0	4	1
P3	0	6	3

- **At Time 0:** All processes arrive at the same time. **P3** has the highest priority (Priority 3), so it starts execution.
- **At Time 6: P3** completes execution. Among the remaining processes, **P1** (Priority 2) has a higher priority than **P2**, so **P1** starts execution.
- **At Time 13: P1** completes execution. The only remaining process is **P2** (Priority 1), so it starts execution.
- **At Time 17: P2** completes execution. All processes are now finished.

- Now, lets calculate average waiting time and turn around time:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	7	13	13	6
P2	0	4	17	17	13
P3	0	6	6	6	0

- **Average Turnaround Time =** 12
- **Average Waiting Time =** 6.33

Example of Preemptive Priority Scheduling (Different Arrival Time)

Process	Arrival Time	Burst Time	Priority
P1	0	6	2
P2	1	4	3
P3	2	5	1

- **At Time 0:** Only P1 has arrived, so it starts execution.
- **At Time 1:** P2 arrives with a higher priority (Priority 3) than P1. P1 is preempted, and P2 starts execution.

- **At Time 5:** P2 completes execution. Both P1 and P3 are available. P1 has the higher priority (Priority 2), so it starts execution.
- **At Time 10:** P1 completes execution. P3 resumes execution to finish its remaining burst time.
- **At Time 15:** P3 completes execution. All processes are now finished.
- Now, lets calculate average waiting time and turn around time:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	6	10	10	4
P2	1	4	5	4	0
P3	2	5	15	13	8

- **Average Turnaround Time =** 9
- **Average Waiting Time =** 4

Non-Preemptive Priority Scheduling

- In Non-Preemptive Priority Scheduling, the CPU is not taken away from the running process. Even if a higher-priority process arrives, the currently running process will complete first.
- Ex: A high-priority process must wait until the currently running process finishes.
- **Example of Non-Preemptive Priority Scheduling:**

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	6	3

1. **At Time 0:** Only P1 has arrived. P1 starts execution as it is the only available process, and it will continue executing till t = 4 because it is a non-preemptive approach.
2. **At Time 4:** P1 finishes execution. Both P2 and P3 have arrived. Since P2 has the highest priority (Priority 1), it is selected next.
3. **At Time 6:** P2 finishes execution. The only remaining process is P3, so it starts execution.
4. **At Time 12:** P3 finishes execution.

Now, lets calculate average waiting time and turn around time:

Note: Lower number represents higher priority.

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	4	4	4	0
P2	1	2	6	5	3
P3	2	6	12	10	4

- **Average Turnaround Time =** 6.33
- **Average Waiting Time =** 2.33

Round Robin(RR) Scheduling

- In this scheduling algorithm, the OS defines a quantum time or a fixed time period.
- And every job is run cyclically for this predefined period of time, before being preempted for the next job in the ready queue.
- The jobs that are preempted before completion go back to the ready queue to wait their turn.
- It is also referred to as the preemptive version of the FCFS scheduling algorithm in OS.
- Once a job begins running, it is executed for a predetermined time and gets preempted after the time quantum is over.
- It is easy and simple to use or implement.
- The RR scheduling algorithm is one of the most commonly used CPU scheduling algorithms in OS.

Advantages of RR Scheduling Algorithm

- This seems like a fair algorithm since all jobs get equal time CPU.
- Does not lead to any starvation problems.
- New jobs are added at the end of the ready queue and do not interrupt the ongoing process.
- Leads to efficient utilization of the CPU.

Disadvantages of RR Scheduling Algorithm

- Every time job runs the course of quantum time, a context switch happens. This adds to the overhead time, and ultimately the overall execution time.
- A low slicing time may lead to low CPU output.
- Important tasks aren't given priority.
- Choosing the correct time quantum is a difficult job.

Scenario 1: Processes with Same Arrival Time

- Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2 ms**

Process	Burst Time	Arrival Time
P1	4 ms	0 ms
P2	5 ms	0 ms
P3	3 ms	0 ms

1. **Time 0-2 (P1)**: P1 runs for 2 ms (total time left: 2 ms).
2. **Time 2-4 (P2)**: P2 runs for 2 ms (total time left: 3 ms).
3. **Time 4-6 (P3)**: P3 runs for 2 ms (total time left: 1 ms).
4. **Time 6-8 (P1)**: P1 finishes its last 2 ms.
5. **Time 8-10 (P2)**: P2 runs for another 2 ms (total time left: 1 ms).
6. **Time 10-11 (P3)**: P3 finishes its last 1 ms.
7. **Time 11-12 (P2)**: P2 finishes its last 1 ms

- Now, lets calculate average waiting time and turn around time:

Processes	AT	BT	CT	TAT	WT
P1	0	4	8	8-0 = 8	8-4 = 4
P2	0	5	12	12-0 = 12	12-5 = 7
P3	0	3	11	11-0 = 11	11-3 = 8

- **Average Turn around time** = (8 + 12 + 11)/3 = 31/3 = 10.33 ms
- **Average waiting time** = (4 + 7 + 8)/3 = 19/3 = 6.33 ms

Implementation

1. Create an array **rem_bt[]** to keep track of remaining burst time of processes. This array is initially a copy of bt[] (burst times array)
2. Create another array **wt[]** to store waiting times of processes. Initialize this array as 0.
3. Initialize time : t = 0
4. Keep traversing all the processes while they are not done. Do following for **i'th** process if it is not done yet.

```
If rem_bt[i] > quantum
    t = t + quantum
    rem_bt[i] -= quantum;
Else // Last cycle for this process
    t = t + rem_bt[i];
    wt[i] = t - bt[i]
    rem_bt[i] = 0; // This process is over
```

Scenario 2: Processes with Different Arrival Times

- Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms

1. **Time 0-2 (P1 Executes):**
   - P1 starts execution as it arrives at 0 ms.
   - Runs for 2 ms; remaining burst time = 5 - 2 = 3 ms.
   - **Ready Queue:** [P1].
2. **Time 2-4 (P1 Executes Again):**
   - P1 continues execution since no other process has arrived yet.
   - Runs for 2 ms; remaining burst time = 3 - 2 = 1 ms.
   - P2 arrive at 4 ms.
   - **Ready Queue:** [P2, P1].
3. **Time 4-6 (P2 Executes):**
   - P2 starts execution as it arrives at 4 ms.
   - Runs for 2 ms; remaining burst time = 2 - 2 = 0 ms.
   - P3 arrive at 5ms
   - **Ready Queue:** [P1, P3].
4. **Time 6-7 (P1 Executes):**
   - P1 starts execution.
   - Runs for 1 ms; remaining burst time = 1 - 1 = 0 ms.
   - **Ready Queue:** [P3].
5. **Time 7-9 (P3 Executes):**
   - P3 starts execution.
   - Remaining burst time = 4 - 2 = 2 ms.
   - **Ready Queue:** [P3].
6. **Time 9-11 (P3 Executes Again):**
   - P3 resumes execution and runs for 2 ms and complete its execution
   - Remaining burst time = 2 - 2 = 0 ms.
   - **Ready Queue:** [].

- **Now, lets calculate average waiting time and turn around time:**

Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	7 ms	7 ms	2 ms
P2	6 ms	2 ms	0 ms
P3	11 ms	6 ms	1 ms

- **Average Turn around time** =7+2+6/3=15/3=5ms
- **Average waiting time** = 2+0+1/3=1ms

Implementation:

1. Declare arrival[], burst[], wait[], turn[] arrays and initialize them. Also declare a timer variable and initialize it to zero. To sustain the original burst array create another array (temp_burst[]) and copy all the values of burst array in it.

2. To keep a check we create another array of bool type which keeps the record of whether a process is completed or not. we also need to maintain a queue array which contains the process indices (initially the array is filled with 0).

3. Now we increment the timer variable until the first process arrives and when it does, we add the process index to the queue array.

4. Now we execute the first process until the time quanta and during that time quanta, we check whether any other process has arrived or not and if it has then we add the index in the queue (by calling the fxn. queueUpdation()).

5. Now, after doing the above steps if a process has finished, we store its exit time and execute the next process in the queue array. Else, we move the currently executed process at the end of the queue (by calling another fxn. queueMaintainence()) when the time slice expires.

5. The above steps are then repeated until all the processes have been completely executed. If a scenario arises where there are some processes left but they have not arrived yet, then we shall wait and the CPU will remain idle during this interval.

Multiple-Level Queues Scheduling

- Multilevel Queue(MLQ) CPU scheduling is a type of scheduling that is applied at the operating system level with the aim of sectioning types of processes and then being able to manage them properly.
- As with the MQ Series, processes are grouped into several queues using MLQ based on known parameters such as priority, memory, or type.
- Every queue has its scheduling policy and processes that are in the same queue are very similar too.
- Priorities are assigned to processes based on their type, characteristics, and importance. For example, interactive processes like user input/output may have a higher priority than batch processes like file backups.
- A feedback mechanism can be implemented to adjust the priority of a process based on its behavior over time. For example, if an interactive process has been waiting in a lower-priority queue for a long time, its priority may be increased to ensure it is executed in a timely manner.

Advantages of MLQ Scheduling Algorithm

- Since processes are permanently assigned to their respective queues, the overhead of scheduling is low, as the scheduler only needs to select the appropriate queue for execution.
- The scheduling algorithm provides a fair allocation of CPU time to different types of processes, based on their priority and requirements.
- The scheduling algorithm ensures that processes with higher priority levels are executed in a timely manner, while still allowing lower priority processes to execute when the CPU is idle.
- Priorities are assigned to processes based on their type, characteristics, and importance, which ensures that important processes are executed in a timely manner.

Disadvantages of MLQ Scheduling Algorithm

- Some processes may starve for CPU if some higher priority queues are never becoming empty.
- It is inflexible in nature.
- There may be added complexity in implementing and maintaining multiple queues and scheduling algorithms.

Process	Burst Time	Arrival Time	Queue Number
P1	4 ms	0 ms	1
P2	3 ms	0 ms	2
P3	8 ms	0 ms	1
P4	5 ms	10 ms	1

- At starting, both queues have process so process in queue 1 (P1, P2) runs first (because of higher priority) in the round-robin fashion and completes after 7 units
- Then process in queue 2 (P3) starts running (as there is no process in queue 1) but while it is running P4 comes in queue 1 and interrupts P3 and start running for 5 seconds and
- After its completion P3 takes the CPU and completes its execution.

Implementation

- When a process starts executing the operating system can insert it into any of the above three queues depending upon its **priority** . For example, if it is some background process, then the operating system would not like it to be given to higher priority queues such as queues 1 and 2. It will directly assign it to a lower priority queue i.e. queue 3. Let's say our current process for consideration is of significant priority so it will be given **queue 1** .
- In queue 1 process executes for 4 units and if it completes in these 4 units or it gives CPU for I/O operation in these 4 units then the priority of this process does not change and if it again comes in the ready queue then it again starts its execution in Queue 1.
- If a process in queue 1 does not complete in 4 units then its priority gets reduced and it is shifted to queue 2.
- Above points 2 and 3 are also true for queue 2 processes but the time quantum is 8 units. In a general case if a process does not complete in a time quantum then it is shifted to the lower priority queue.
- In the last queue, processes are scheduled in an FCFS manner.
- A process in a lower priority queue can only execute only when higher priority queues are empty.
- A process running in the lower priority queue is interrupted by a process arriving in the higher priority queue.

Example: Consider a system that has a CPU-bound process, which requires a burst time of 40 seconds. The multilevel Feed Back Queue scheduling algorithm is used and the queue time quantum '2' seconds and in each level it is incremented by '5' seconds. Then how many times the process will be interrupted and in which queue the process will terminate the execution?

Solution:

- Process P needs 40 Seconds for total execution.
- At Queue 1 it is executed for 2 seconds and then interrupted and shifted to queue 2.
- At Queue 2 it is executed for 7 seconds and then interrupted and shifted to queue 3.
- At Queue 3 it is executed for 12 seconds and then interrupted and shifted to queue 4.
- At Queue 4 it is executed for 17 seconds and then interrupted and shifted to queue 5.
- At Queue 5 it executes for 2 seconds and then it completes.
- Hence the process is interrupted 4 times and completed on queue 5.

Assignment 2

- How does process differ from program? Explain process state with the help of block diagram.
- Why some process requires high priority? What would happen if all processes have some the priority? Mention merits and demerits of assigning priority on process.
- Do you think a process can exist without any state? Justify your view with the help of process state transition diagram.
- For each of the following transitions between the processes states, indicate whether the transition is possible. If it is possible, give an example of one thing that would cause it.
  - Running->Ready
  - Running->Blocked
  - Blocked->Running
- How process model is implemented? Explain with diagram.

- When threads are better than processes? Explain the concept of user level threads in detail.
- How thread based execution minimizes the context switching problem of process based execution? Explain the different multithreading model.
- Describe how multithreading improves performance over a singled- threaded solution.
- Explain "race condition" and also state how process synchronization is handled using semaphore? Explain with algorithms.
- What kind of problem arises with sleep and wakeup mechanism of achieving mutual exclusion? Explain with suitable code snippet.
- How semaphore is used for the process synchronization? Do you think semaphore is the best solution for solving critical section problem? Explain using it in Producer-consumer problem.
- "Using semaphore is very critical for programmer" Do you support this statement? If yes, prove the statement with some fact. If not, put your view with some logical facts against the statement.

- Define the term semaphore. How does semaphore help in dining philosophers problem? Explain.
- Explain the Peterson's concept for the solution of critical section problem.
- What is critical section problem? Why executing critical selection must be mutual exclusive? Explain.
- What is lock variable? Discuss its working and problems associated with it in detail.
- Show how sleep and wake up solution is better than busy waiting solution for the critical section problem.
- How Produce-Consumer problem can be solved with sleep and wakeup primitives? Explain.
- Briefly define the term scheduler in the context of any operating system. List three aims of the scheduler in any operating system.
- Differentiate between preemptive and non-preemptive scheduling.
- What is the turnaround time for each algorithm?
- Why some process requires high priority? What would happen if all processes have some the priority? Mention merits and demerits of assigning priority on process.

- Round-robin scheduling behaves differently depending on its time quantum. Can the time quantum be set to make round robin behave the same as any of the following algorithms? If so how? Proof the assertion with an example.
- FCFS
- SJF
- SRTN
- Consider the following set of processes, with the arrival times and the CPU-burst times given in milliseconds.

Process	Arrival time	Burst Time
P1	0 ms	5 ms
P2	1 ms	3 ms
P3	2 ms	3 ms
P4	4 ms	1 ms

a) What is the average turnaround time for these processes with the preemptive Shortest Remaining Processing Time First algorithm ?

b) The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

c) What is the total waiting time for process P2?

- What is race condition? Calculate average waiting and average turnaround time of the given set of processes in table below using SJF and RR scheduling algorithm. [Note: Quantum time for RR=3].

Process id	Arrival Time	Execution Time
A	0	8
B	2	14
C	9	19
D	19	7
E	25	15

- For the processes listed in following table, draw a Gantt chart illustrating their execution using:
  a) First-come-First-Serve
  b) Short-Job-First
  c) Shortest-Remaining-Time-Next
  d) Round-Robin (quantum=2)
  e) Round-Robin (quantum=1)

Processes	Arrival Time	CPU Time
A	0.000	3
B	1.001	6
C	4.001	4
D	6.002	2

- Defined interactive system goals? List various interactive scheduling algorithms. Consider following process data and compute average waiting time and average turnaround time for RR(quantum 10) and priority scheduling algorithms.

PID	Burst Time	Arrival Time	Priority
A	16	0	1
B	37	12	2
C	25	7	3